



INSTITUTO TÉCNICO LA FALDA
CUEANEXO - 140293100

INFORME DE PROYECTO OLIMPIADAS INET 2023

Alarma código azul

CUE

Nº 140293100

Área

Programación

Curso

7º Año 2023

Localidad

La Falda - Córdoba

Integrantes

Antún, Lucas Santiago Said - 3548 63-6430 - est.antun.lucas@latecnicalf.com.ar

Martinovic, Jovan - 3548 40-0949 - est.martinovic.jovan@latecnicalf.com.ar

Martínez Jansen, Leonel - 3548 40-8760 - est.martinez.leonel@latecnicalf.com.ar

Rodriguez, Mateo Ariel - 3548 61-1783 - est.rodriguez.mateo@latecnicalf.com.ar

Docente a cargo

Ruíz, Anabel

Título de Grado - Ingeniera en Sistemas de Información

Cargo Docente - Bases de datos I, II

Email - ruiz.anabel@latecnicalf.com.ar

Tel. - 11 4079-8329

Índice

Índice	2
1. Alarma Código Azul - Introducción	3
2. Modelo de Requisitos y análisis	4
2.1 Análisis	4
2.2 Historias de usuario	7
2.3 Diagramas y diseños	11
Primer modelo Diagrama de Entidad Relación	11
2.4 Actores y roles	13
3. Planificación y organización	14
Las buenas prácticas implementadas:	14
4. Desarrollo	15
4.1 Planning del sistema	15
4.2 Desarrollo y diseño de vistas	15
4.3 Desarrollo de funcionalidades	16
4.4 Desarrollo de aplicación móvil	16
4.5 Funcionamiento del sistema realizado en Cisco Packet Trace	17
5. Testeos algoritmos, funcionalidades e interacciones	18
6. Errores y soluciones	18
Problemática 1	19
Problemática 2	20
Problemática 3	21
Problemática 4	21
7. Alternativas posibles	23
8. Anexo imágenes	34
9. Registro de experiencia	35
10. Bibliografía	36

1. Alarma Código Azul - Introducción

El presente proyecto tiene como objetivo desarrollar un sistema de gestión y reportes estadísticos para emergencias médicas, denominado "Alarma Código Azul". El sistema está destinado a optimizar el procedimiento de atención de emergencias médicas en entidades hospitalarias públicas.

Contexto

La Municipalidad de una ciudad del interior de la provincia de Tucumán elaboró una licitación pública destinada a empresas PYME de la provincia. El alcance técnico de la misma está dado por el desarrollo e implementación de un sistema destinado a dotar de la máxima eficiencia al procedimiento de atención de las emergencias médicas denominadas "ALARMA CÓDIGO AZUL".

Motivación

El desarrollo de un sistema de gestión y reportes estadísticos para emergencias médicas es una necesidad urgente. El sistema permitirá optimizar el procedimiento de atención de emergencias médicas, lo que redundará en una mejora de la calidad de atención a los pacientes.

Objetivos

Los objetivos del proyecto son los siguientes:

- Desarrollar un sistema de gestión y reportes estadísticos para emergencias médicas que cumpla con los requerimientos técnicos de la licitación pública.
- Implementación del sistema de gestión nombrado.

Metodología de desarrollo

El proyecto se desarrollará utilizando la metodología ágil "Scrum". Scrum es una metodología que permite desarrollar software de alto impacto de manera flexible y adaptable.

Importancia

El desarrollo de un sistema de gestión y reportes estadísticos para emergencias médicas es una contribución importante para la salud pública. El sistema permitirá mejorar la calidad de atención a los pacientes, lo que redundará en una reducción de la mortalidad y morbilidad.

Datos adicionales

- El sistema se desarrolló en Python con el framework de Django, con una base de datos hecha en sqlite3, junto con HTML, CSS y JS.
- La aplicación móvil se realizó en Flutter Flow.
- El sistema se desarrolló utilizando el sistema de control de versiones Git, alojando el código fuente en GitHub.
- Hemos utilizado Visual Studio Code como editor de código.

2. Modelo de Requisitos y análisis

En este apartado nos centraremos en mostrar en forma de diagrama: modelo relacional, users stories y la definición de los actores.

2.1 Análisis

En un comienzo, no teníamos mucha idea de cómo era un sistema de emergencias, por lo que nos pusimos en contacto con diferentes entes hospitalarios de la zona, gerentes y allegados a este tipo de lugares. De los cuales pudimos adentrarnos en una sala de emergencias que nos permitió tener un concepto aún más claro, por ello confeccionamos una lista en la que obtuvimos datos realmente importantes para poder desarrollar correctamente el sistema.

Triage

El triage (también conocido como clasificación de pacientes) es un proceso utilizado en entornos de atención médica para determinar la prioridad de atención que deben recibir los pacientes según la gravedad de su condición y los recursos disponibles. El objetivo principal del triage es asegurar que los pacientes más críticamente enfermos o heridos reciban atención inmediata, mientras que aquellos con afecciones menos graves puedan esperar de manera segura.

El proceso de triage implica la evaluación rápida de los pacientes para determinar la gravedad de su condición. Esto se hace utilizando un sistema de clasificación que generalmente se basa en la severidad de la lesión o enfermedad, la estabilidad del paciente y la probabilidad de recuperación.

Los pacientes se clasifican en diferentes categorías, que pueden variar según el sistema utilizado, pero generalmente incluyen:

- **Prioridad 1 o Roja (Emergencia):** Pacientes con afecciones que amenazan la vida y que requieren atención inmediata.
- **Prioridad 2 o Amarilla (Urgente):** Pacientes con afecciones graves pero no inmediatamente amenazantes para la vida. Pueden necesitar atención en un plazo corto.
- **Prioridad 3 o Verde (Semi-Urgente):** Pacientes con afecciones que no son graves y que pueden esperar un poco más para recibir atención.
- **Prioridad 4 o Azul (No Urgente):** Pacientes con afecciones no urgentes que pueden esperar sin riesgo para su salud.
- **Prioridad 5 o Blanca (No Urgente):** Llamados de pacientes, provenientes de algún box, la llamada puede ser originada desde el baño o desde la cama, con afecciones no urgentes que pueden esperar sin riesgo para su salud.

El triage es una herramienta esencial para los profesionales de la salud en situaciones de emergencia para garantizar que los recursos se utilicen de manera efectiva y que se atienda a los pacientes de acuerdo con la gravedad de su situación.

Áreas de una sala de emergencias

Con la recopilación de información obtuvimos que las salas de emergencia cuentan con diferentes salas que son comunes en los centros de atención inmediata de los distintos establecimientos.

Las diferentes salas que se seleccionaron son:

- **Boxes:** son espacios individuales o habitaciones pequeñas designadas para la evaluación y tratamiento de pacientes. Cada box generalmente está equipado con una cama o camilla, suministros médicos básicos y monitores para controlar signos vitales. Los profesionales de la salud evalúan y proporcionan atención inicial a los pacientes en estos espacios. Los boxes ofrecen privacidad y permiten que el personal médico se enfoque en el paciente de manera más eficaz.
- **Camas Frías (Cold Beds):** Son camas que no están siendo utilizadas en ese momento y están disponibles para admitir a un paciente que necesite atención médica inmediata. Esto es especialmente importante en situaciones de emergencia donde el tiempo es crucial y se requiere un acceso rápido a la atención médica.
- **Sala de Shock Room o Sala de Shock:** La "Sala de Shock" es un área especializada en una sala de emergencias diseñada para tratar pacientes en estado de shock. Esta área está equipada con equipos avanzados de monitorización y resucitación para proporcionar atención de emergencia a pacientes en condiciones críticas.
- **Sala de Traumatismos:** La "Sala de Traumatismos" es un espacio en la sala de emergencias especialmente equipado para el tratamiento de pacientes con lesiones traumáticas graves. Puede contar con recursos adicionales, como equipos de radiología y quirúrgicos, para manejar lesiones graves como fracturas complejas, heridas penetrantes y otros tipos de traumas graves.
- **Quirófano:** Aunque no es parte de una sala de emergencias típica, pero los centros más modernos, al menos cuentan con una. Los quirófanos son espacios dedicados a realizar procedimientos quirúrgicos. En situaciones de emergencia, los pacientes que requieren cirugía inmediata pueden ser trasladados a un quirófano adyacente a la sala de emergencias. Esto permite la realización de procedimientos quirúrgicos de manera rápida y eficaz en situaciones críticas.
- **Laboratorios:** Un laboratorio en una sala de emergencias, a menudo conocido como laboratorio de urgencias, cumple la función de realizar pruebas y análisis de laboratorio de manera rápida y eficiente para apoyar en el diagnóstico y tratamiento de pacientes que se encuentran en situación de emergencia.

Personal de una sala de emergencias

Las salas de trata inmediata trabajan en conjunto para proporcionar atención de emergencia efectiva y oportuna a los pacientes que llegan a la sala de emergencias. Trabajan en un ambiente de alta presión y deben estar preparados para tomar decisiones rápidas y precisas para salvar vidas.

- **Recepcionista** (Encargado/a de la Sala de Emergencias): El recepcionista en una sala de emergencias es el primer punto de contacto para los pacientes y sus familias al llegar al departamento de emergencias. Su función es registrar la información básica del paciente, como nombre, motivo de la visita y detalles de contacto. También pueden realizar una evaluación inicial de la gravedad de la situación para determinar la prioridad de atención. El recepcionista coordina el flujo de pacientes y puede asignarlos a boxes o áreas específicas según la gravedad de su condición.

Médicos: En una sala de emergencias, hay varios tipos de médicos que pueden estar presentes:

- **Médicos de Emergencia:** Son médicos entrenados para evaluar y tratar una amplia variedad de emergencias médicas y lesiones. Toman decisiones rápidas sobre la atención y pueden ordenar pruebas diagnósticas y tratamientos.

- **Médicos Especialistas:** Dependiendo de la complejidad de los casos, pueden ser llamados especialistas como cirujanos, cardiólogos, ortopedistas, etc., para proporcionar una atención más especializada.
- **Pacientes:** Los pacientes son las personas que acuden a la sala de emergencias buscando atención médica urgente debido a una lesión, enfermedad o condición médica aguda. Pueden variar desde personas con afecciones menos graves hasta aquellas en situaciones de emergencia que requieren tratamiento inmediato.
- **Enfermeros/as:** Los enfermeros en la sala de emergencias juegan un papel crucial en el cuidado y la atención de los pacientes. Sus responsabilidades incluyen:
 - Realizar evaluaciones iniciales de los pacientes.
 - Administrar medicamentos y tratamientos.
 - Realizar procedimientos como extracciones de sangre o suturas.
 - Monitorear los signos vitales y la condición de los pacientes.
 - Coordinar el flujo de pacientes y la comunicación entre el personal médico.

2.2 Historias de usuario

Para poder comprender un poco más de las funcionalidades, veremos algunas de las historias de usuario que realizamos para poder comenzar a desarrollar el sistema en cuestión.

Inicio de Sesión Seguro y Personalizado para Médicos, Administradores y Recepcionistas

Descripción: Como usuario registrado (ya sea médico, administrador o recepcionista), deseo poder acceder a mi cuenta proporcionada por el director del hospital a través de un proceso de inicio de sesión. Este proceso deberá utilizar mi dirección de correo electrónico y contraseña previamente registrados para garantizar la seguridad y la autenticación adecuada.

Criterios de Aceptación:

- El sistema debe proporcionar una página de inicio de sesión que permita a los usuarios acceder a sus cuentas de manera segura.
- Los usuarios deben seleccionar el tipo de cuenta (médico, administrador o recepcionista) al iniciar sesión.
- El formulario de inicio de sesión debe solicitar la dirección de correo electrónico y la contraseña previamente registradas.
- El sistema debe validar la autenticación del usuario antes de permitir el acceso a la cuenta.
- En caso de credenciales incorrectas, se deberá mostrar un mensaje de error claro y coherente.
- Después de una autenticación exitosa, el usuario será redirigido a la página principal correspondiente a su rol (médico, administrador o recepcionista).
- El sistema debe cifrar y proteger la información de inicio de sesión y cumplir con las mejores prácticas de seguridad de datos.

Registro de Llamados a Pacientes y Gestión de Triage

Descripción: Como administrador o recepcionista, necesito una página dedicada para registrar diferentes tipos de llamados de emergencia basados en el proceso de Triage. El Triage es un método que me permite gestionar el riesgo clínico y determinar con precisión el nivel de emergencia de los pacientes, especialmente cuando la demanda supera los recursos disponibles. Esta función se integra con el famoso sistema de colores de Triage para proporcionar clasificaciones detalladas.

Criterios de Aceptación:

- La plataforma debe proporcionar una página dedicada para el registro de llamadas de pacientes.
- El sistema debe permitir la clasificación de los llamados en función de los diferentes colores del Triage, indicando el nivel de emergencia.
- El administrador o recepcionista será responsable de cargar los llamados a través de la interfaz, proporcionando información detallada sobre la situación del paciente y el nivel de emergencia.
- La aplicación móvil destinada a los médicos permitirá la visualización de los llamados creados por el recepcionista de turno en tiempo real, asegurando una respuesta rápida y eficiente.
- El sistema deberá contar con un historial completo que permita a los médicos revisar los llamados previamente atendidos, incluyendo detalles sobre el diagnóstico y el tratamiento proporcionado.
- Se deben implementar notificaciones para alertar a los médicos sobre nuevos llamados y actualizaciones relevantes.
- Se deberá garantizar la privacidad y confidencialidad de la información del paciente en todo momento, cumpliendo con las regulaciones de protección de datos.

Administración de Áreas en un Entorno Hospitalario

Descripción: Como usuario del sistema, deseo tener la capacidad de gestionar las diversas áreas de un hospital mediante un sistema de Alta, Baja y Modificación (ABM). Esto me permitirá cargar, editar y eliminar áreas específicas, facilitando así la organización y administración eficiente de los recursos hospitalarios. Las áreas a gestionar incluyen "Boxes" (por ejemplo, Box 1, Box 2), Salas de Traumatismos, Salas de Shockroom, Camas Frías y Áreas de Traumatismos.

Criterios de Aceptación:

- El sistema debe proporcionar una interfaz de usuario intuitiva y fácil de usar para la gestión de áreas hospitalarias.
- El usuario debe tener la capacidad de crear, eliminar y editar las diferentes áreas, proporcionando información detallada como nombre y ubicación física.
- El usuario debe poder modificar los detalles de un área existente, incluyendo su nombre y ubicación.

- Se debe incluir una funcionalidad para eliminar áreas, asegurando que no haya dependencias con pacientes o registros médicos asociados antes de completar la operación.
- Se debe proporcionar retroalimentación clara al usuario después de cada operación (alta, baja o modificación), confirmando el éxito o informando sobre posibles errores.

Dashboard de Análisis para Evaluación de Llamadas

Descripción: Como usuario autenticado en el sistema, requiere acceso a un dashboard que proporcione estadísticas y análisis relevantes sobre los llamados realizados. Esto me permitirá tomar decisiones fundamentadas basadas en el rendimiento y la eficacia de las llamadas. El dashboard debe ofrecer visualizaciones claras y detalladas para una comprensión efectiva.

Criterios de Aceptación:

- La plataforma debe proporcionar un dashboard accesible para usuarios autenticados, con información relevante sobre las llamadas realizadas.
- El dashboard debe incluir un gráfico de barras que muestre el promedio de respuestas exitosas por día en el último mes, facilitando la visualización de tendencias a lo largo del tiempo.
- Debe existir un gráfico circular que desglose el tipo de respuestas (por ejemplo, emergencia o genérico) para proporcionar una visión detallada de la distribución de los llamados.
- El dashboard debe mostrar el promedio de duración de las llamadas en minutos y segundos, proporcionando información valiosa sobre la eficiencia de la atención.

Visualización y Detalle de Información de Pacientes, Exportación de Registros

Descripción: En la sección de pacientes, se debe ofrecer la posibilidad de visualizar todos los pacientes registrados por la recepcionista al generar los llamados o cuando ingresan a la sala de emergencias. La página mostrará información esencial, incluyendo patologías, datos personales y el médico que atendió su llamado. Además, se debe habilitar la opción de exportar un registro individual de cada paciente en formatos PDF o CSV. Al seleccionar un paciente, se debe permitir el acceso a información aún más detallada.

Criterios de Aceptación:

- El sistema debe proporcionar una sección de pacientes accesible para los usuarios autorizados.
- Se debe mostrar una lista completa de pacientes, incluyendo información relevante como patologías, datos personales y médico que atendió el llamado o ingreso a la sala de emergencias.

- Cada paciente debe contar con la opción de exportar su registro en formatos PDF o CSV para facilitar el almacenamiento y el análisis de datos.
- Al seleccionar un paciente, se debe ofrecer acceso a una vista detallada con información más exhaustiva, que incluya antecedentes médicos, tratamientos previos y cualquier otra información pertinente.
- La interfaz de la sección de pacientes debe ser intuitiva y fácil de navegar para brindar una experiencia de usuario óptima.
- La exportación de registros debe garantizar la precisión y completitud de la información proporcionada en el archivo PDF o CSV.
- Se debe implementar un mecanismo de seguridad que permita el acceso solo a usuarios autorizados, garantizando la confidencialidad de la información del paciente.
- Se debe proporcionar una opción de búsqueda y filtrado para facilitar la localización de pacientes específicos.

Acceso al Historial de Pacientes Dados de Alta en la Sala de Emergencias

Descripción: En la sección de pacientes, se debe habilitar un botón que permita al usuario acceder al historial de pacientes que han sido dados de alta en la sala de emergencias. Esto facilitará la visualización de datos específicos sobre dichos pacientes.

Criterios de Aceptación:

- La plataforma debe contar con un botón dedicado que permite acceder al historial de pacientes dados de alta en la sala de emergencias.
- Al hacer clic en el botón, se debe presentar una lista completa de los pacientes que han sido dados de alta, junto con información detallada sobre cada uno.
- La información detallada debe incluir datos específicos sobre cada paciente, como diagnósticos, tratamientos proporcionados y recomendaciones para el seguimiento posterior al alta.
- La interfaz de visualización del historial de pacientes debe ser intuitiva y fácil de navegar, proporcionando una experiencia de usuario óptima.
- Se debe garantizar la confidencialidad y seguridad de la información del paciente, cumpliendo con las regulaciones de protección de datos.

Visualización y Gestión de Médicos con Datos y Especialidades Detalladas

Descripción: En la sección de médicos, se debe permitir a los usuarios ver la lista de médicos disponibles junto con su información personal y especialidad. Esto incluye datos como nombre, contacto y especialización. Además, se debe habilitar la gestión de médicos, lo que implica la posibilidad de agregar, eliminar y modificar la información de estos profesionales.

Criterios de Aceptación:

- La plataforma debe proporcionar una sección dedicada para visualizar la lista de médicos disponibles.

- Para cada médico, se deben mostrar detalles como nombre completo, información de contacto y su especialización médica.
- Los usuarios deben tener la capacidad de agregar nuevos médicos al sistema, proporcionando información completa sobre el profesional.
- Se debe permitir la eliminación de médicos del sistema, con la debida confirmación para evitar eliminaciones accidentales.
- Los usuarios deben poder editar la información de un médico existente, lo que incluye la capacidad de actualizar detalles como especialización, horario de trabajo y datos de contacto.
- La interfaz de gestión de médicos debe ser intuitiva y fácil de usar para garantizar una experiencia de usuario eficiente.
- Se debe garantizar la confidencialidad y seguridad de la información del médico, cumpliendo con las regulaciones de protección de datos.

Administración de Usuarios y Asignación de Roles por el Superusuario

En la sección de usuarios, el superusuario o administrador del sistema tendrá la capacidad de gestionar y administrar las cuentas de acceso al sistema. Esto incluye la posibilidad de asignar roles específicos a cada usuario, como médico o recepcionista. El superusuario será el único autorizado para llevar a cabo esta gestión de roles.

Criterios de Aceptación:

- La plataforma debe proporcionar una sección dedicada para la administración de usuarios, accesible únicamente por el superusuario o administrador del sistema.
- Se debe mostrar una lista completa de usuarios registrados, incluyendo detalles como nombre, cargo y dirección de correo electrónico.
- El superusuario debe tener la capacidad de asignar roles específicos a cada usuario, como médico o recepcionista, según las necesidades del sistema.
- Se debe permitir al superusuario agregar nuevos usuarios al sistema, proporcionando información completa sobre cada persona.
- Debe ser posible eliminar cuentas de usuarios, con la debida confirmación para evitar eliminaciones accidentales.
- El superusuario debe poder editar la información de los usuarios existentes, incluyendo la capacidad de actualizar detalles como cargo, dirección de correo electrónico y asignación de roles.
- La interfaz de gestión de usuarios debe ser intuitiva y fácil de usar para garantizar una experiencia de usuario eficiente.
- Se debe garantizar la confidencialidad y seguridad de la información de los usuarios, cumpliendo con las regulaciones de protección de datos.

2.3 Diagramas y diseños

Primer modelo Diagrama de Entidad Relación

Un Diagrama de Entidad-Relación (DER), es una herramienta de modelado que permite la gestión de bases de datos. Su propósito principal es representar de manera gráfica la estructura de una base de datos, enfocándose en las relaciones entre diferentes entidades y cómo se relacionan entre sí.

Un Diagrama ER se compone de tres elementos principales:

Entidades: Representan los objetos o conceptos del mundo real que se almacenan en la base de datos.

Atributos: Son las propiedades o características que describen una entidad.

Relaciones: Indican cómo las entidades están conectadas entre sí. Pueden ser de diferentes tipos, como relaciones uno a uno, uno a muchos o muchos a muchos. Se representan con líneas que conectan las entidades y se etiquetan para especificar la naturaleza de la relación.

Conforme el desarrollo surgía, captamos que necesitábamos aún muchas más cosas como nuevas tablas, relaciones y atributos. Pese a los errores, este fue el diagrama que pudimos confeccionar:

- **Figura 2.**

Un Modelo Relacional es un enfoque teórico y matemático para representar y organizar datos en una base de datos orientado al personal de desarrollo.

En un modelo relacional, los datos se organizan en tablas o relaciones. Cada tabla representa una entidad y sus atributos, y está compuesta por filas y columnas. A cada columna se le asigna un nombre y un tipo de dato específico, como texto, número, fecha, etc.

Las características principales de un modelo relacional son:

Tablas: Cada entidad se representa como una tabla, donde las filas representan las instancias individuales de esa entidad y las columnas representan los atributos.

Claves Primarias: Cada tabla tiene una clave primaria, que es un campo o combinación de campos que identifica de manera única cada fila en la tabla. Por ejemplo, en una tabla de pacientes, la clave primaria podría ser el número de identificación del paciente.

Relaciones: Las relaciones entre tablas se establecen a través de claves foráneas. Una clave foránea es un campo en una tabla que se relaciona con la clave primaria de otra tabla. Esto permite establecer conexiones entre diferentes entidades.

Integridad Referencial: Esta es una propiedad importante en un modelo relacional. Garantiza que las relaciones entre tablas se mantengan de manera consistente, lo que significa que no se pueden crear relaciones entre registros que no tengan correspondencia en la tabla relacionada.

El modelo relacional que obtuvimos en base a los puntos anteriores fue el siguiente:

- [Figura 1](#)

Finalmente el modelo relacional final obtenido, que fue utilizado para para el desarrollo es el que se verá a continuación:

- [Figura 3](#)

2.4 Actores y roles

Nombre del Actor	Rol del Actor
Médico	El rol de médico en nuestro sistema es poder visualizar las llamadas de los pacientes que se realicen, así como también, observar datos de los pacientes y emitir reportes de los mismos.
Recepcionista a cargo	Este rol será el encargado de cargar todos los datos de los pacientes, médicos y áreas al sistema, realizar los llamados, emitir reportes pudiendo visualizar cualquier tipo de información, ya sea de pacientes, áreas o médicos.
Administrador del sistema	El administrador tendrá todos los permisos, como novedad, podrá visualizar, así como gestionar los distintos roles del sistema, ya sea médico o recepcionista, en otras palabras cumplirá el rol de superusuario.
Enfermeros	Al igual que el médico, el enfermero/a podrá visualizar las llamadas de los pacientes que se realicen, así como también, observar datos de los pacientes y emitir reportes de los mismos.

3. Planificación y organización

En esta etapa continuamos con la aplicación del proceso ágil, siguiendo con los fundamentos de la metodología SCRUM para resolver la complejidad de implementar y producir con calidad el sistema correspondiente a la materia. SCRUM permite definir y verificar requisitos captados en etapas anteriores, analizar y diseñar con orientación a objetos, planificar, estimar, organizar y guiar el desarrollo del proyecto, trabajar con suficiente detalle y precisión, y responder adecuadamente frente a demandas cambiantes del negocio y los usuarios del sistema. Trabajando con recursos tales como los Roles, las Épicas, las Historias de Usuario, el Product Backlog y sus elementos priorizados, los Releases y sus Sprints, y los Entregables.

Las buenas prácticas implementadas:

- **Product Backlog:** lista priorizada de necesidades del sistema.
- **Sprint:** ciclo de tiempo para la entrega de valor.
- **Daily Scrum:** reuniones cortas del equipo para comunicación rápida (qué hice ayer, qué haré hoy, qué impedimentos tengo).
- **Historias de usuarios:** descripción a alto nivel de la necesidad del usuario.
- **Retrospectiva:** reuniones especiales para analizar y mejorar la forma de trabajo del equipo.

Los roles de SCRUM que utilizaremos en el desarrollo de sistema son:

- INET, encargada del tiempo de entrega y las consignas pautadas.
- Scrum Team:
 - Antún, Lucas Santiago Said.
 - Martinovic, Jovan.
 - Martínez Jansen, Leonel Francisco.
 - Rodríguez, Mateo Ariel.

Para la gestión de este proyecto se utilizarán las siguientes herramientas:

- ✓ Visual Studio Code
- ✓ Sqlite 3.
- ✓ Python (Django framework).
- ✓ Flutter Flow.

El Tablero de trabajo se continuará gestionando en Trello, con un formato como el siguiente:

Anexo Imágenes - [Figura 4](#)

4. Desarrollo

En este punto se verán los procedimientos llevados a cabo en la confección del sistema.

4.1 Planning del sistema

En el planning, consideramos todos los puntos claves para poder desarrollar el sistema, esto incluye las vistas, el tipo de base de datos, el lenguaje de programación controladores, entre otras cosas.

Estos fueron los principales:

- **Vistas:** Las vistas son las pantallas que verán los usuarios del sistema donde se debe definir el diseño y la funcionalidad de cada vista.
- **Base de datos:** La base de datos es el lugar donde se almacenarán los datos del sistema, donde, la estructura de las tablas y las relaciones entre las tablas es una función clave a la hora de realizar consultas y realizar controladores.

- **Lenguaje de programación:** El lenguaje de programación es el lenguaje que se utilizará para desarrollar el sistema, en este caso nos decidimos por usar Django, un framework de desarrollo web escrito en Python, la decisión se llevó a cabo debido a que ya tenemos conocimiento previos en el lenguaje y conocemos eficiencia a las que nos podía brindar el lenguaje.
- **Funciones o controladores:** Los controladores son los componentes que se encargan de procesar las peticiones de los usuarios y de actualizar la base de datos.

Además de estos aspectos, es claro destacar:

- **Cronograma:** El cronograma debe indicar las fechas de inicio y finalización de cada tarea, así como el tiempo estimado para cada tarea y/o funcionalidad. La gestión de tareas y tiempo es indispensable para poder desarrollar trabajos o proyectos de este calibre.

4.2 Desarrollo y diseño de vistas

En este apartado nos encargamos del diseño de las plantillas, así como el desarrollo de vistas del nuestro sistema

En una primera instancia decidimos crear plantillas con la herramienta de Canva, que nos permitió diseñar modelos a seguir para el desarrollo de la visualización final del usuario que utilice nuestro sistema.

1. Login. [Figura 5](#)
2. Llamados a médicos. [Figura 6](#)
3. Reportes de llamadas. [Figura 7](#)
4. Dashboard con datos estadísticos. [Figura 8](#)
5. Gestión de salas ABM (Alta, baja y modificación). [Figura 9](#)
6. Agregado de salas. [Figura 10](#)
7. Gestión de médicos ABM (Alta, baja y modificación). [Figura 11](#)
8. Agregado de médicos. [Figura 12](#)
9. Gestión de pacientes ABM (Alta, baja y modificación). [Figura 13](#)
10. Reportes de pacientes. [Figura 14](#)
11. Gestión de usuarios del sistema ABM (Alta, baja y modificación). [Figura 15](#)

4.3 Desarrollo de funcionalidades

En el proceso de desarrollo del sistema, se prioriza la eficiencia y la optimización del código para garantizar un rendimiento óptimo y una mantenibilidad sostenible a largo plazo. Para lograr este objetivo, se emplearon funciones específicas creadas en Python utilizando el potente marco de trabajo Django.

Cada controlador y función fue diseñado para cumplir con los requisitos precisos del sistema, utilizando las herramientas y características proporcionadas por Django. Esto incluyó la utilización de clases y métodos genéricos, así como el aprovechamiento de las capacidades de administración de nuestra base de datos integrada

Además, se implementaron técnicas de modularización para garantizar la cohesión y el acoplamiento adecuado entre las diferentes partes del sistema. Esto

facilita la extensibilidad y la adaptabilidad del código a medida que evoluciona el proyecto.

El uso de pruebas unitarias y funcionales fue fundamental para verificar el correcto funcionamiento de cada controlador y función, asegurando así la confiabilidad y la estabilidad del sistema en todo momento.

4.4 Desarrollo de aplicación móvil

En el proceso de creación de la aplicación móvil, se optó por la versatilidad y la eficiencia que ofrece FlutterFlow como plataforma de desarrollo. Esta elección estratégica ha permitido construir una interfaz dinámica y altamente adaptable para los profesionales médicos..

La interfaz de la aplicación se ha diseñado de manera intuitiva, brindando a los doctores acceso rápido y sencillo a los llamados de los pacientes. La disposición de elementos y la navegación se han optimizado para garantizar una experiencia fluida y eficaz.

Uno de los aspectos destacados de FlutterFlow es su capacidad para integrar notificaciones en tiempo real de manera eficaz. Esto permite a los doctores recibir actualizaciones instantáneas sobre nuevas solicitudes y cambios relevantes en el sistema..

El enfoque en la usabilidad y la experiencia del usuario ha sido prioritario en el desarrollo de la aplicación móvil. FlutterFlow ha facilitado la creación de una interfaz intuitiva y fácil de usar, lo que garantiza que los médicos puedan acceder a la información de manera rápida, sencilla y eficiente.

4.5 Funcionamiento del sistema realizado en Cisco Packet Trace

Para poder modelar la funcionalidad del sistema, implementamos la siguiente herramienta “Cisco Packet Tracer” es un programa de simulación de redes desarrollado por Cisco Systems. Está diseñado para ayudar a los estudiantes y profesionales de redes a comprender y practicar conceptos de redes de computadoras. Packet Tracer proporciona una plataforma virtual en la cual los usuarios pueden crear, configurar y solucionar problemas de redes complejas sin necesidad de hardware físico.

Esta topología representa un sistema de control y monitoreo de dispositivos, en nuestro caso, la sala de emergencias:

- **DLC 100 Home Gateway:** Este es un dispositivo central que actúa como un punto de acceso y control para los diferentes dispositivos de la red. Puede proporcionar servicios como la gestión de conexiones inalámbricas, seguridad y asignación de direcciones IP.
- **Dispositivos (Alarma y Luces):** Son los elementos que se conectan al DLC 100 Home Gateway. En este caso, la alarma y las luces están distribuidas en diferentes salas y se conectan de manera inalámbrica. Esto significa que utilizan tecnologías como Wi-Fi o Bluetooth para comunicarse con el Gateway.

- **Conexiones inalámbricas:** Las conexiones inalámbricas permiten la comunicación entre el DLC 100 Home Gateway y los dispositivos como la alarma y las luces. Estas conexiones se basan en tecnologías inalámbricas estándar y seguras.
- **Botón Accionador de la Alarma:** Este botón está conectado a un MCU-PT (unidad de control multipunto). El MCU-PT probablemente se utiliza para manejar múltiples dispositivos y señales, y en este caso, se encarga de recibir la señal del botón y transmitirla al DLC 100 Home Gateway.
- **Sala de Operaciones** o lugar donde se encuentra el/la Recepcionista: Estos son los centros de control desde donde se supervisa y se realizan los llamados, que posteriormente harán sonar la alarma y las luces. Desde aquí se puede interactuar con el DLC 100 Home Gateway.

En otras palabras, la topología proporciona una infraestructura para controlar de manera centralizada una alarma y luces distribuidas en diferentes salas. El DLC 100 Home Gateway actúa como un punto central de comunicación, y las conexiones inalámbricas permiten la comunicación entre el Gateway y los dispositivos. El botón accionador de la alarma, conectado a un MCU-PT, permite activar la alarma cuando sea necesario. La sala de operaciones o la recepcionista tienen la capacidad de supervisar y controlar todos estos elementos desde su ubicación.

- Diseño obtenido: [Figura 16](#)

5. Testeos algoritmos, funcionalidades e interacciones

Debido a inconvenientes al intentar hostear el sistema, dado que no se logró abordar a tiempo el hosteo, se muestra un video para exhibir todo el sistema, el cual es ejecutado de un localhost del framework de django.

- Video del sistema: https://www.youtube.com/watch?v=wT7_rz6IRCo
- Video de la aplicación móvil: <https://www.youtube.com/shorts/5rqQNn9gy7k>
- Repositorio del proyecto en github:

<https://github.com/PDEjovan/olimpiadasInet>

6. Errores y soluciones

Problemática 1:

La situación que se presenta es la necesidad de realizar una operación delicada en el entorno de desarrollo, específicamente en el repositorio alojado en GitHub. El motivo subyacente es la detección de problemas relacionados con las migraciones de la base de datos. Esto implica que se requiere una acción más radical, como el borrado completo de la base de datos existente y su posterior recreación.

Análisis de la situación:

El proceso de borrado y recreación de una base de datos es una tarea crítica que debe llevarse a cabo con sumo cuidado. Implica una serie de etapas, cada una con sus propias implicaciones y riesgos potenciales. El primer y más esencial paso es

la realización de un respaldo exhaustivo de la base de datos actual. Este paso no debe omitirse bajo ninguna circunstancia, ya que garantiza la preservación de los datos cruciales en caso de cualquier eventualidad durante el proceso.

Estrategia de ejecución:

El procedimiento se inicia con el acceso al repositorio en GitHub y la plataforma de desarrollo asociada. Aquí, se busca y modifica la configuración de la base de datos, identificando la opción para eliminarla. Una vez completado este paso, es imperativo asegurarse de que todas las configuraciones relevantes estén correctamente establecidas para crear una nueva base de datos. Esto puede implicar detalles como el nombre de la base de datos, las credenciales de acceso y el tipo de base de datos.

Con la base de datos borrada y las configuraciones actualizadas, se procede a crear una nueva instancia de la base de datos. A continuación, se aplican las migraciones pertinentes a esta nueva base de datos. Si se ha realizado un respaldo de datos, este es el momento de importarlos de vuelta a la base de datos recién creada.

Verificación y Pruebas:

El siguiente paso es crítico para garantizar la integridad y la funcionalidad del sistema. Se realiza una exhaustiva verificación del funcionamiento de la aplicación con la nueva base de datos. Esto incluye la ejecución de pruebas detalladas para asegurar que todas las características y funcionalidades operen como se espera. Cualquier anomalía o error debe ser tratado con prioridad y resuelto antes de proceder.

Comunicación y Documentación:

Si se trabaja en un entorno colaborativo, es crucial comunicar los cambios realizados en la base de datos al equipo. Esto puede implicar una reunión o notificación detallada, proporcionando información sobre los pasos tomados y las implicaciones para otros miembros del equipo. Además, si existe documentación relacionada con la base de datos, esta debe ser actualizada para reflejar los cambios realizados.

Consideraciones Finales:

Es importante destacar que este proceso debe ser ejecutado con la máxima precaución y preferiblemente en un entorno de desarrollo o pruebas, antes de aplicarse en el entorno de producción. Adicionalmente, se recomienda mantener un registro detallado de todas las acciones realizadas, lo que facilitará la identificación y solución de problemas futuros.

Problemática 2:

Durante el proceso de desarrollo de software, es común enfrentarse a una serie de errores que afectan la integridad y la estructura de la base de datos. Estos errores pueden surgir debido a diversos factores, como fallos en la lógica de programación, cambios en los requisitos del sistema o actualizaciones en las bibliotecas y frameworks utilizados. Estos inconvenientes pueden manifestarse en forma de inconsistencias en los datos, relaciones incorrectas, o incluso en la pérdida parcial de información. Es fundamental abordar estos problemas de manera efectiva y rápida para mantener la integridad y el funcionamiento adecuado del sistema en desarrollo.

Análisis de la situación:

La detección de errores en la base de datos es un aspecto crítico del proceso de desarrollo. Estos errores pueden ser sutiles y difíciles de identificar, o pueden presentarse como fallos evidentes en la funcionalidad de la aplicación. El primer paso en esta situación es realizar un análisis exhaustivo para determinar la naturaleza y el alcance del problema. Esto implica revisar los registros de errores, examinar el código fuente relacionado con la base de datos y, en algunos casos, llevar a cabo pruebas específicas para replicar y comprender el problema.

Estrategia de ejecución:

Una vez que se han identificado los errores en la base de datos, es esencial abordarlos de manera sistemática y precisa. Esto puede implicar una serie de acciones, dependiendo de la naturaleza del error:

Corrección de la lógica de programación:

- Si el error es causado por una lógica de programación incorrecta, se debe modificar el código relevante para garantizar la correcta interacción con la base de datos.

Actualización de las migraciones:

- En casos donde los errores están relacionados con las migraciones de la base de datos, se deben crear nuevas migraciones o modificar las existentes para corregir los problemas.

Revisión de las relaciones de la base de datos:

- Si se detectan problemas en las relaciones entre las tablas, es necesario ajustar las claves primarias y extranjeras para garantizar la integridad de los datos.

Reconstrucción de la base de datos (si es necesario):

- En situaciones críticas, puede ser necesario realizar una operación más drástica, como la reconstrucción completa de la base de datos. Esto debe hacerse con extrema precaución y siempre después de haber realizado un respaldo completo de los datos.

Verificación y Pruebas:

Después de realizar las correcciones, es crucial llevar a cabo una verificación exhaustiva. Esto implica probar cada aspecto de la aplicación que está vinculado a la base de datos, asegurándose de que los errores han sido completamente resueltos y que no se han introducido nuevas inconsistencias.

Comunicación y Documentación:

Es esencial mantener a todos los miembros del equipo informados sobre los cambios realizados en la base de datos debido a errores de desarrollo. Además, cualquier documentación relevante debe actualizarse para reflejar las modificaciones realizadas.

Consideraciones Finales:

La gestión efectiva de los errores en la base de datos es esencial para mantener la integridad y funcionalidad del sistema en desarrollo. La comunicación

clara y la documentación precisa son pilares fundamentales para asegurar que todos los miembros del equipo estén al tanto de los cambios y puedan trabajar de manera colaborativa para resolver los problemas.

Problemática 3:

En el proceso de desarrollo de la aplicación, se ha identificado una situación en la que algunos campos en la base de datos han tenido que ser configurados como "nulos" (permitiendo valores nulos) debido a restricciones que impedían la modificación de la base de datos. Esta decisión puede tener implicaciones en la integridad de los datos y debe ser tratada con precaución.

Análisis de la situación:

La necesidad de permitir valores nulos en ciertos campos de la base de datos puede surgir de restricciones de la plataforma, la arquitectura de la base de datos o de la forma en que se interactúa con la misma desde la aplicación. Esta situación puede presentarse como una solución temporal para evitar conflictos y permitir la continuación del desarrollo, pero requiere una atención especial para garantizar que no se introduzcan datos inconsistentes o nulos de manera inadvertida.

Estrategia de ejecución:

Para abordar esta situación, se deben seguir los siguientes pasos:

Revisar y documentar los campos nulos:

- Es importante identificar y documentar los campos en la base de datos que han sido configurados como "null". Esto proporcionará una visión clara de qué campos están afectados y por qué se les ha permitido contener valores nulos.

Validar la entrada de datos:

- Cuando un campo se configura como "null", se debe implementar una validación estricta en la aplicación para asegurarse de que no se ingresen valores nulos de manera inadvertida. Esto puede incluir controles en formularios y validaciones en el código.

Implementar reglas de negocio:

- Se deben establecer reglas de negocio claras sobre cuándo y cómo se permiten valores nulos en estos campos. Esto ayudará a evitar situaciones donde los datos nulos puedan causar problemas de integridad.

Establecer procedimientos de actualización:

- Si en el futuro se desea modificar estos campos para no permitir valores nulos, se deben establecer procedimientos claros para realizar esta operación de manera segura, lo que podría implicar la necesidad de realizar una migración de la base de datos.

Verificación y Pruebas:

Después de implementar las medidas mencionadas, es crucial llevar a cabo pruebas exhaustivas para asegurarse de que la aplicación funcione correctamente y que no se introduzcan valores nulos de manera inadvertida en los campos que se han configurado de esa manera..

Comunicación y Documentación:

Es fundamental comunicar estos cambios al equipo de desarrollo y documentar las razones detrás de la configuración de campos. Además, se debe mantener un registro detallado de cualquier modificación realizada en la estructura de la base de datos.

Consideraciones Finales:

La configuración de campos puede ser una solución temporal válida, pero requiere una gestión cuidadosa para evitar la introducción de datos inconsistentes o nulos. La comunicación clara y la documentación adecuada son esenciales para asegurar que todos los miembros del equipo estén al tanto de los cambios y las razones detrás de ellos.

Problemática 4:

Análisis de la Situación:

La problemática radica en la presencia de errores durante el desarrollo de la experiencia móvil. Esto indica que el proceso de desarrollo no está alcanzando los resultados deseados y que se requiere una solución que permita al menos crear una demo funcional. Para abordar esta situación, se ha optado por utilizar FlutterFlow, una herramienta que simplifica el desarrollo de aplicaciones móviles con Flutter.

Estrategia de Ejecución:

Evaluación de Errores: El primer paso es identificar y analizar los errores específicos que están surgiendo durante el desarrollo. Esto puede requerir la revisión del código, la identificación de inconsistencias o la depuración de problemas técnicos, tales como errores de llamados de firebase, o bien algún problema de alguna plantilla, variables y demás.

Integración de FlutterFlow: Se procede a utilizar FlutterFlow para crear una versión funcional de la aplicación. FlutterFlow proporciona una interfaz gráfica que facilita la construcción de la interfaz de usuario y la lógica de la aplicación, lo que puede acelerar el proceso de desarrollo.

Desarrollo de una Demo Funcional: Utilizando FlutterFlow, se enfoca en construir una versión simplificada pero funcional de la aplicación móvil. Esto implica la implementación de las características clave que permitan demostrar el propósito y la funcionalidad de la aplicación.

Pruebas y Verificación: Una vez completada la demo funcional, se procede a realizar pruebas exhaustivas para asegurarse de que la aplicación cumple con los requisitos mínimos de funcionamiento. Esto incluye la verificación de la interfaz de usuario, la funcionalidad de las características y la detección y corrección de posibles errores.

Consideraciones Finales:

Es importante recordar que FlutterFlow es una herramienta que puede facilitar el desarrollo, pero no reemplaza la comprensión y el conocimiento de Flutter y la programación móvil en general. Si surgen problemas más complejos o personalizados, puede ser necesario recurrir a soluciones más avanzadas en Flutter o en alguna otra tecnología.

Problemática 5:

Debido a que trabajamos con el framework de Django del cual no conocíamos al 100% nos surgió un inconveniente al momento de hostear ya que debería de ser realizado con un Ubuntu Server, el cual no sabemos cómo crear, y debido al poco tiempo restante para la entrega. Concluimos que lo mejor sería grabar videos del funcionamiento cuando lo ejecutamos con el Localhost de Django. Para así poder mostrar cómo se ve el sistema. En cuanto a los videos se encuentran en la carpeta del proyecto y también se encuentra un link de youtube que muestra el sistema en el apartado de este documento de Testeos algoritmos, funcionalidades e interacciones.

7. Alternativas posibles

Alternativas de desarrollo

Inicialmente, se contempló la posibilidad de crear una aplicación de escritorio, cuyo desarrollo implicaría el aprovechamiento de bibliotecas renombradas como pygame y tkinter, entre otras opciones potenciales. Sin embargo, después de un análisis de las alternativas posibles, se tomó la decisión de orientar el proyecto hacia el desarrollo de una página web. Este cambio de enfoque se fundamentó en una serie de consideraciones estratégicas y prácticas, que incluyen la accesibilidad universal que ofrece una plataforma web, la facilidad de actualizaciones y la posibilidad de alcanzar una audiencia más amplia y diversa. Además, la elección de una página web proporciona una mayor flexibilidad en términos de compatibilidad con diferentes dispositivos y sistemas operativos, lo que es esencial para garantizar una experiencia para los usuarios.

Al evaluar diferentes enfoques para la gestión del sistema, es esencial considerar diversas alternativas que se ajusten a las necesidades y contextos específicos. A continuación, se presentan algunas opciones viables, cada una con sus propias ventajas y consideraciones, para abordar eficazmente la implementación del sistema.

Alternativas de gestión del sistema

Opción elegida: Triage de emergencias

La opción que elegimos fue el triage de emergencias. Esta decisión se basó en nuestro conocimiento de las instalaciones locales y en la investigación que llevamos a cabo en los hospitales de la zona, donde pudimos constatar que este sistema estaba implementado. Esto también facilita la transición y la implementación del sistema de manera más eficaz.

Alternativas posibles

1. Clasificación de Emergencias según el Sistema START:

- Verde (Caminando): Pacientes con heridas leves o sin heridas, pueden caminar y no necesitan atención inmediata.
- Amarillo (Observación): Pacientes con heridas o problemas de salud que necesitan atención médica, pero no en el momento.
- Rojo (Atención Inmediata): Pacientes con afecciones graves que necesitan atención médica inmediata.

- Negro (Fallecido o no recuperable): Pacientes fallecidos o con heridas tan graves que no se espera que sobrevivan incluso con atención médica.

2. Clasificación de Emergencias según el Protocolo PHTLS:

- **Nivel 1** (Amenaza inmediata para la vida): Pacientes con problemas de salud o lesiones que amenazan inmediatamente su vida y requieren intervención inmediata.
- **Nivel 2** (Potencialmente amenaza para la vida): Pacientes con problemas de salud o lesiones que podrían amenazar la vida si no se abordan rápidamente.
- **Nivel 3** (Lesiones o enfermedades que requieren atención): Pacientes con lesiones o enfermedades que requieren atención médica, pero que no son amenazantes para la vida.

Alternativas en tecnologías de desarrollo

- **¿FlutterFlow o React Native?**

La elección de FlutterFlow como plataforma para la creación de la interfaz móvil se fundamentó en su capacidad excepcional para agilizar el proceso de desarrollo y proporcionar una experiencia de usuario de alta calidad. FlutterFlow ofrece una interfaz gráfica intuitiva y una amplia gama de widgets personalizables que permiten una construcción rápida y eficiente. Además, su enfoque en el código único para múltiples plataformas elimina la necesidad de desarrollar y mantener dos códigos base distintos. Esta ventaja significativa, combinada con la robustez y estabilidad que FlutterFlow ofrece, hizo que fuera la elección natural sobre React Native.

- **¿Django o Laravel?**

La elección de Django sobre Laravel para el desarrollo del backend se basó en varios factores clave. Django, conocido por su estructura sólida y su enfoque en la simplicidad y productividad, se destacó por su capacidad para acelerar el proceso de desarrollo y proporcionar una base robusta para la aplicación. Su administrador de bases de datos integrado y su sistema de administración de contenido facilitaron la gestión y organización de datos de manera eficiente. Además, la amplia comunidad y la abundancia de bibliotecas y paquetes disponibles en Django fueron recursos valiosos para la implementación de funcionalidades específicas. Siendo un marco de trabajo altamente escalable y confiable,

Además, la experiencia previa en Python también fue un factor determinante en la elección de Django. El equipo ya contaba con un sólido conocimiento en este lenguaje de programación, lo que facilitó la adopción del framework y aceleró el proceso de desarrollo. Esta familiaridad con Python permitió al equipo aprovechar al máximo las capacidades y características del marco de trabajo, optimizando así la eficiencia en la creación de controladores y funciones del sistema.

8. Anexo imágenes

Figura 1

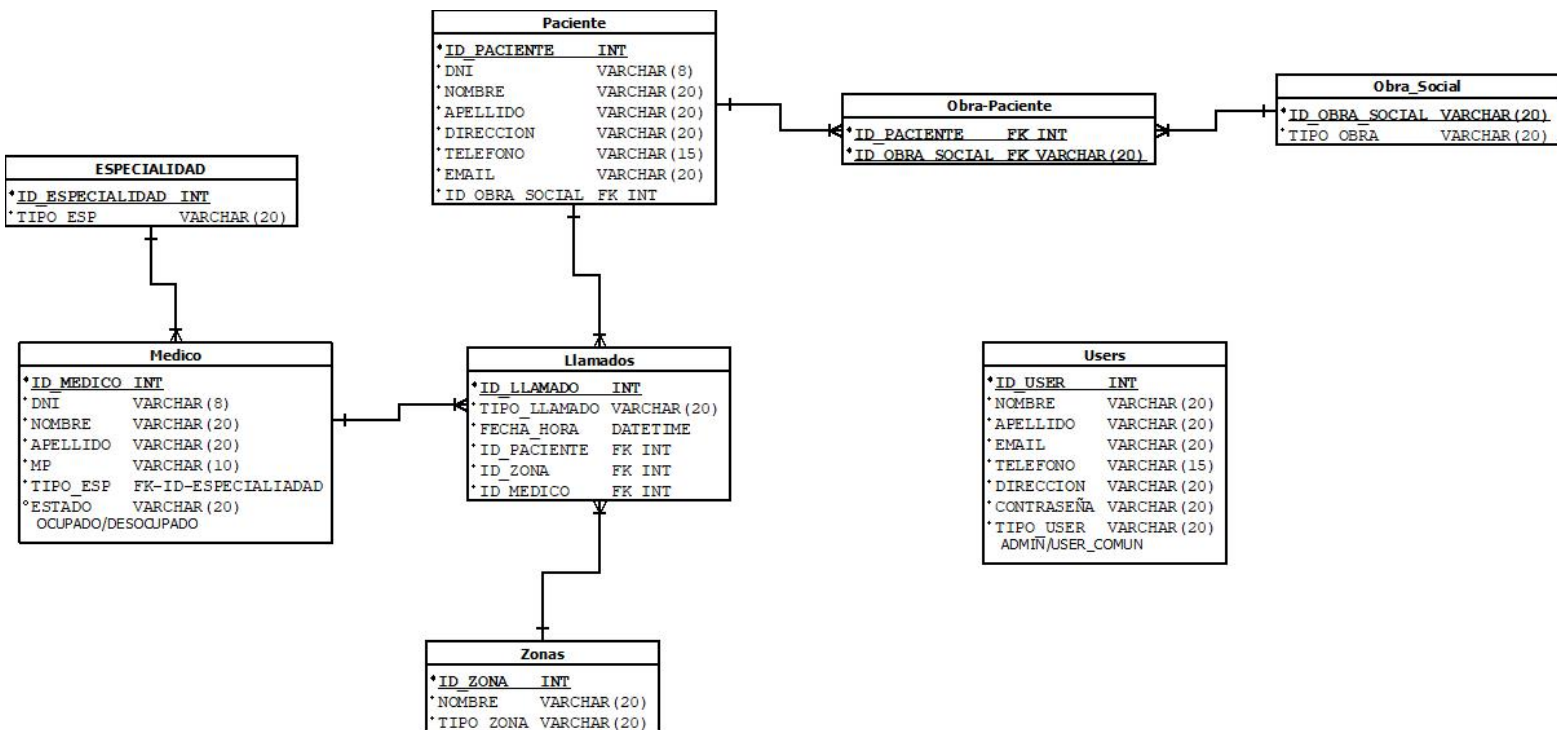


Figura 2

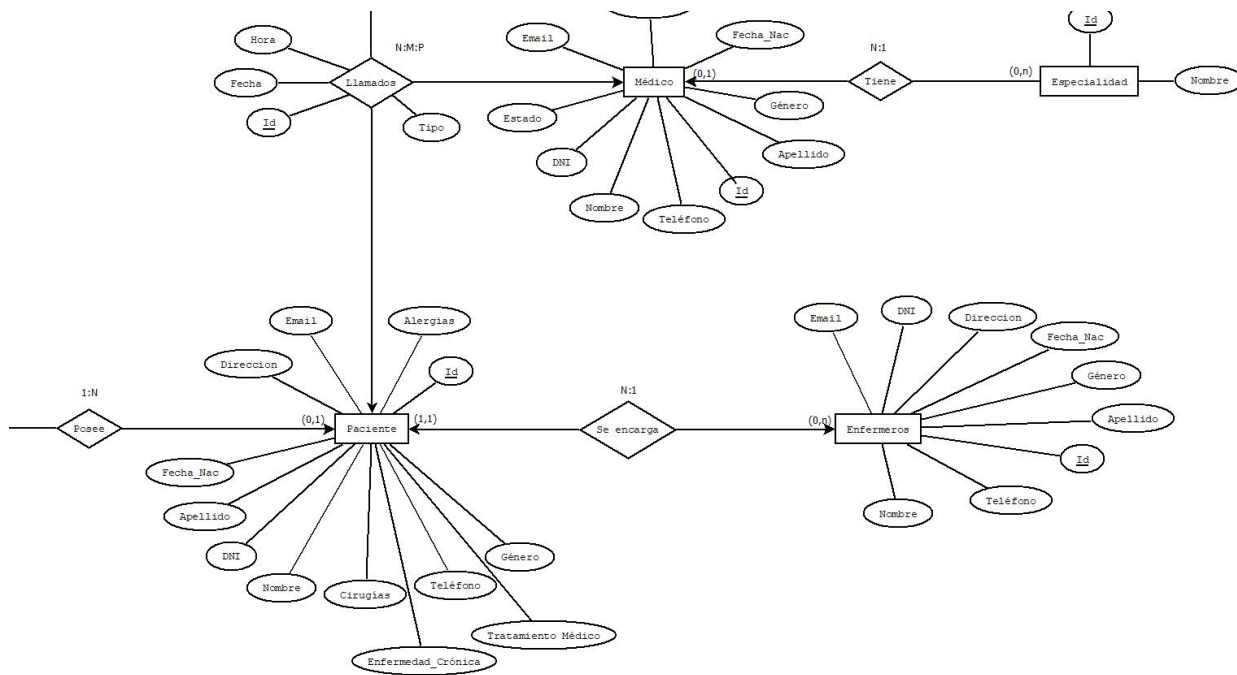


Figura 3

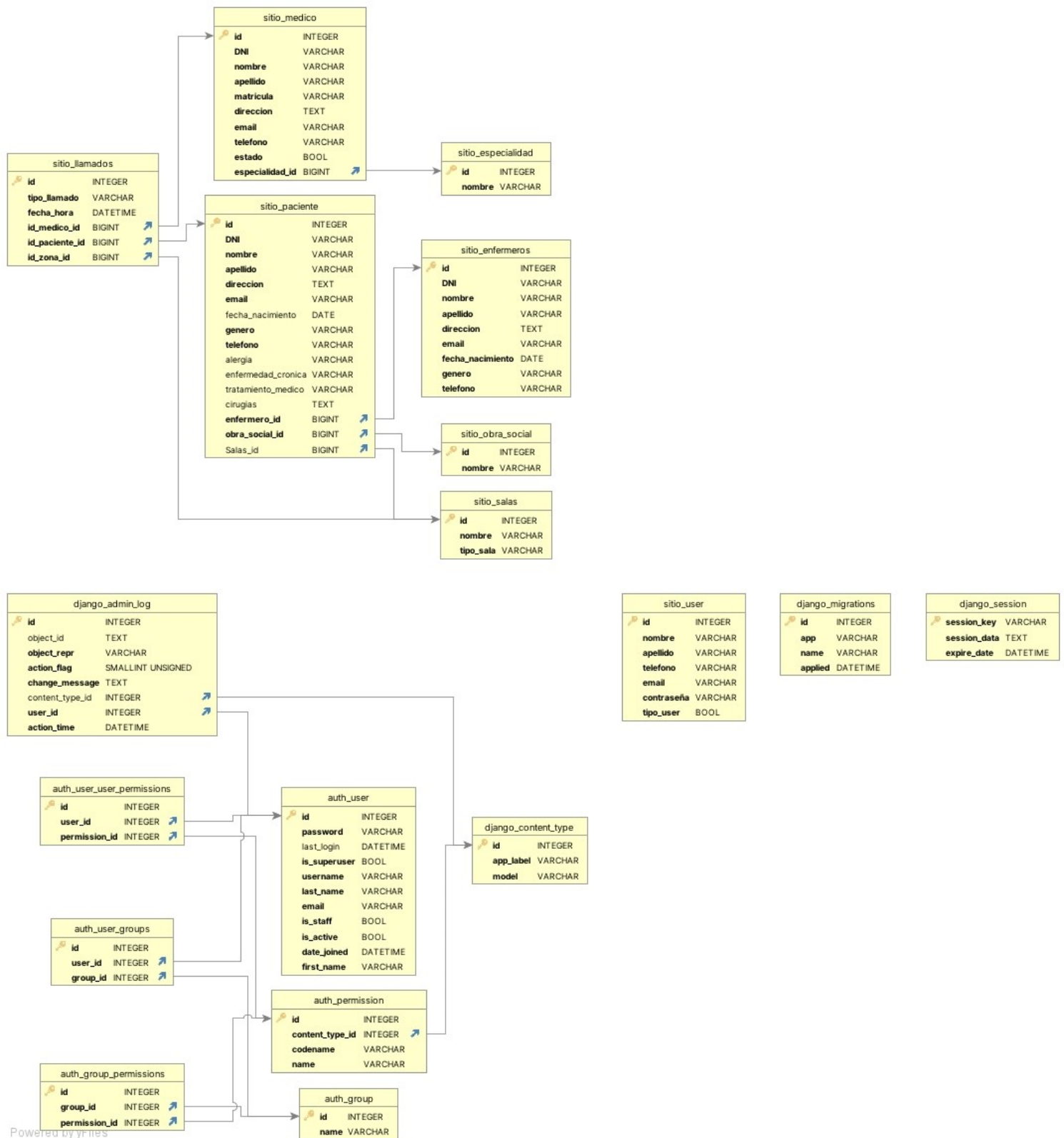


Figura 4

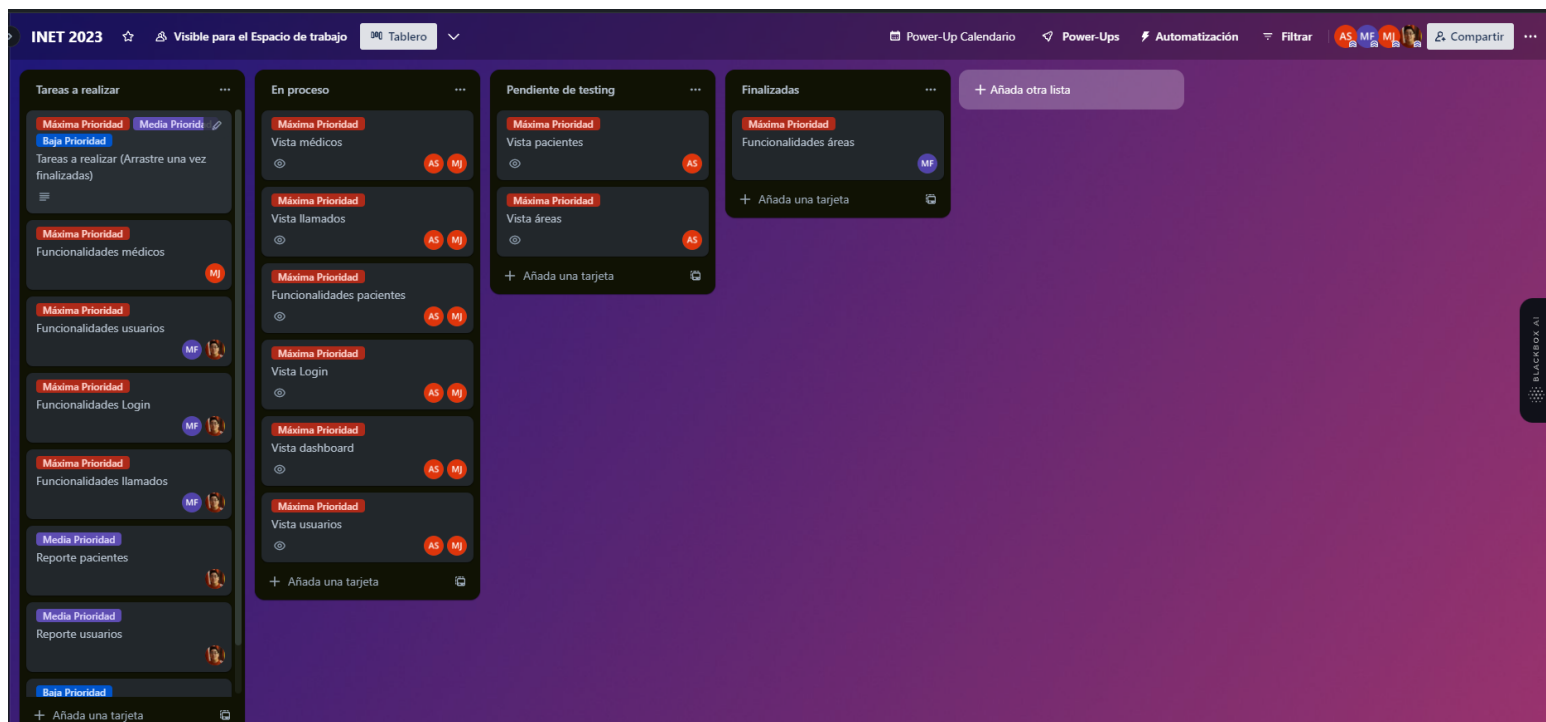


Figura 5



Figura 6

Llamados

▼ Claudia Alves

Buscar...

Llamados finalizados

Ordenar por Nombre

Llamados pacientes

Asistencia	Emergencia	Asistencia
Sala 8 - (Cama) Paciente Carlos Perez Atender Ver más	Sala 8 - (Baño) Paciente Saul Menem Atender Ver más	Sala 8 - (Cama) Paciente Carlos Perez Atender Ver más
Sala 8 - (Cama) Paciente Carlos Perez Atender Ver más	Sala 8 - (Cama) Paciente Carlos Perez Atender Ver más	Sala 8 - (Cama) Paciente Saul Menem Atender Ver más
Sala 8 - (Cama) Paciente Carlos Perez Atender Ver más	Sala 8 - (Baño) Paciente Carlos Perez Atender Ver más	Sala 8 - (Cama) Paciente Carlos Perez Atender Ver más

Admin

Menu

Llamados

Preference

My Profile

Settings

Figura 7

Llamados

▼ Claudia Alves

Llamado 12341

Generar Reporte [CSV](#) [PDF](#)

#ID 12341

Reporte

Año - 2023	Nombre - Carmen
Mes - 09	Apellido - Díaz
Día - 12	Género - Femenino
Hora - 16:43	
Origen - Baño	Médico - Armando Paredes
Estado - Finalizada	Esp - Ginecólogo

Admin

Menu

Llamados

Preference

My Profile

Settings

Figura 8

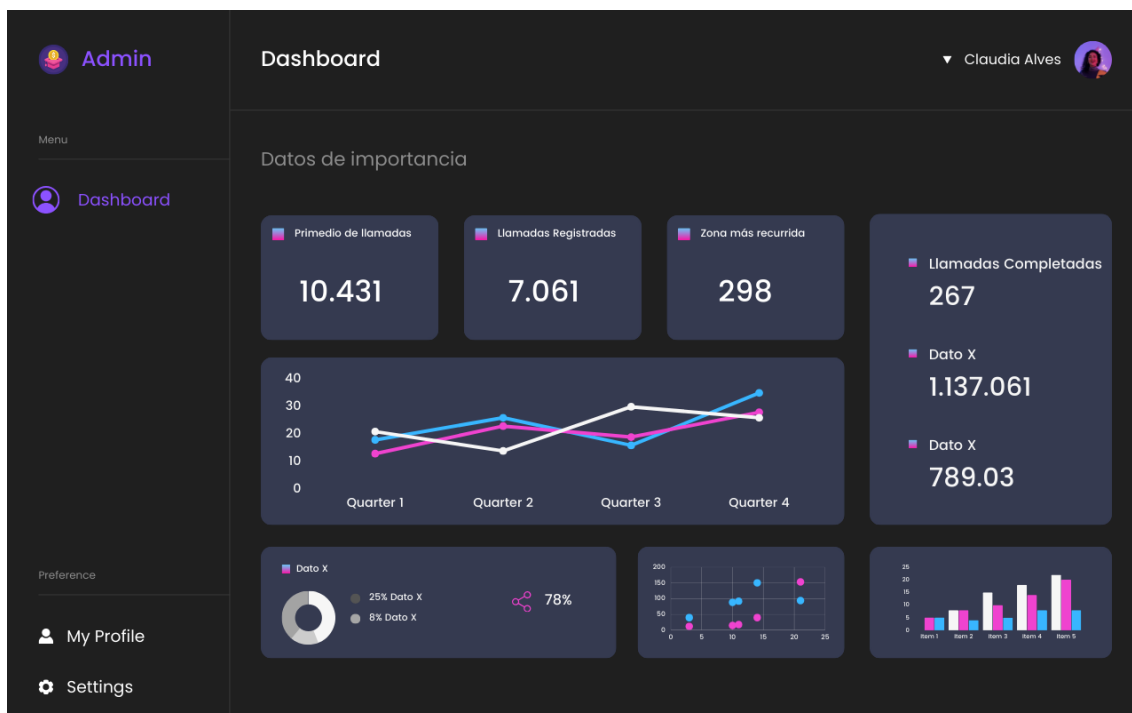


Figura 9

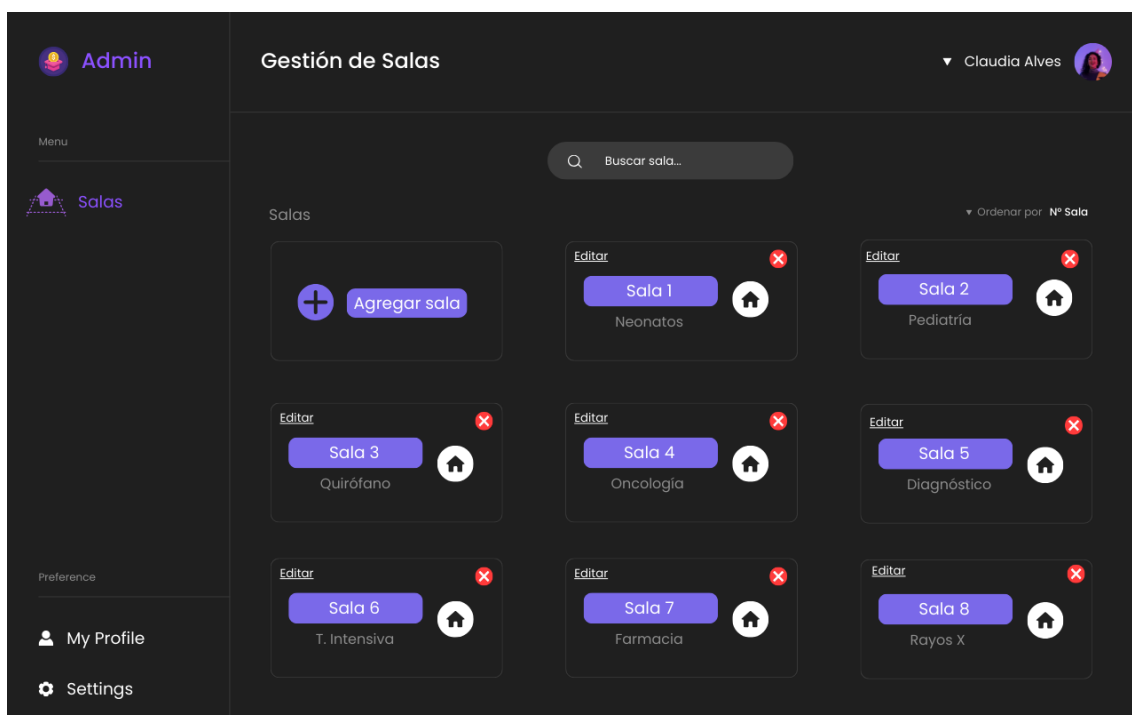


Figura 10

Gestión de salas

Admin

Menu

Salas

Preference

My Profile

Settings

▼ Claudia Alves

Agregar sala

Nro. sala:

Tipo Sala:

Agregar >

Figura 11

Gestión de Médicos

Admin

Menu

Medicos

Preference

My Profile

Settings

▼ Claudia Alves

Buscar médicos...

Médicos

Ordenar por: DNI

Agregar médico

Editar

Agusto Pérez

Anestestesista

DNI 12345

Editar

Romina Díaz

Cirujana

DNI 12345

Editar

NyAp

Especialidad

DNI 12345

Editar

NyAp

Especialidad

DNI 12345

Editar

NyAp

Especialidad

DNI 12345

Editar

NyAp

Especialidad

DNI 12345

Editar

NyAp

Especialidad

DNI 12345

Editar

NyAp

Especialidad

DNI 12345

Figura 12

**Admin**

Menu

**Medicos**

Preference

My Profile

Settings

Gestión de Médicos



Agregar médico

Nombre

Apellido

DNI

MP

Especialidad

Calle

Altura

Agregar

▼

Claudia Alves

Figura 13

 Admin

Menu

 Pacientes

Preference

 My Profile

 Settings

Gestión de Pacientes

Claudia Alves



Buscar pacientes...

Pacientes

Ordenar por DNI



Agregar médico

Editor

NyAp

Sala X

DNI 12345



Editor

NyAp

Sala X

DNI 12345



Editor

NyAp

Sala X

DNI 12345



Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

Editor

NyAp

Sala X

DNI 12345

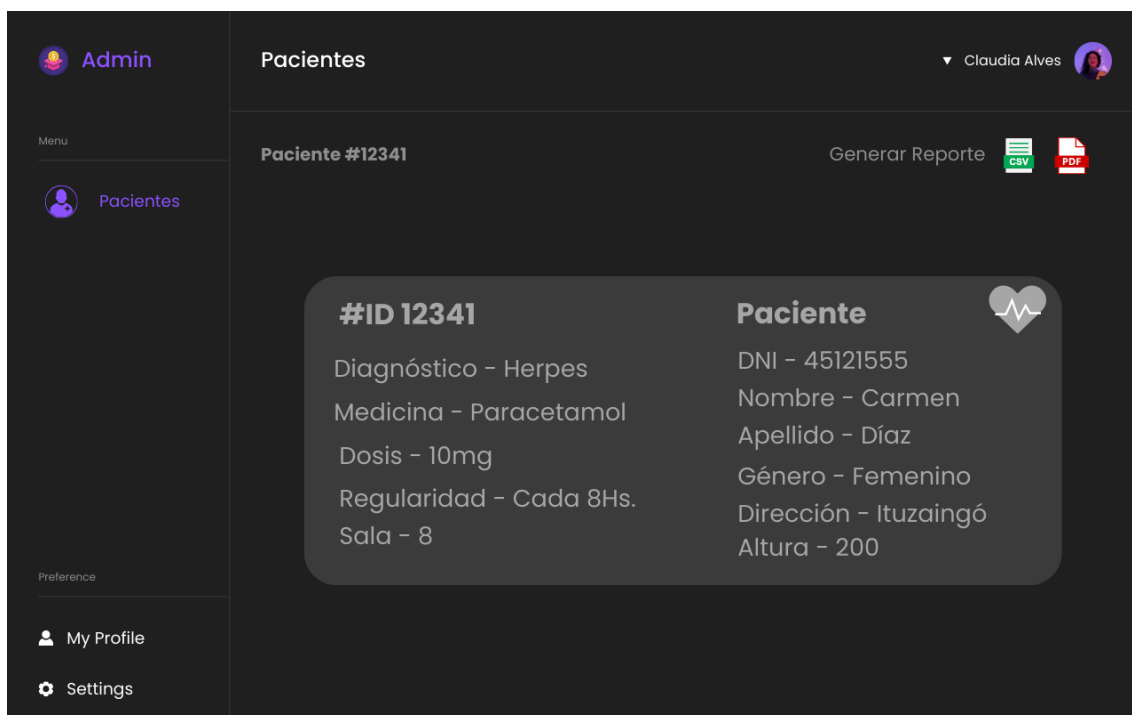
Editor

NyAp

Sala X

DNI 12345

Figura 14



Pacientes

▼ Claudia Alves

Menu

Pacientes

Paciente #12341

Generar Reporte CSV PDF

#ID 12341

Diagnóstico - Herpes

Medicina - Paracetamol

Dosis - 10mg

Regularidad - Cada 8Hs.

Sala - 8

Paciente

DNI - 45121555

Nombre - Carmen

Apellido - Díaz

Género - Femenino

Dirección - Ituzaingó

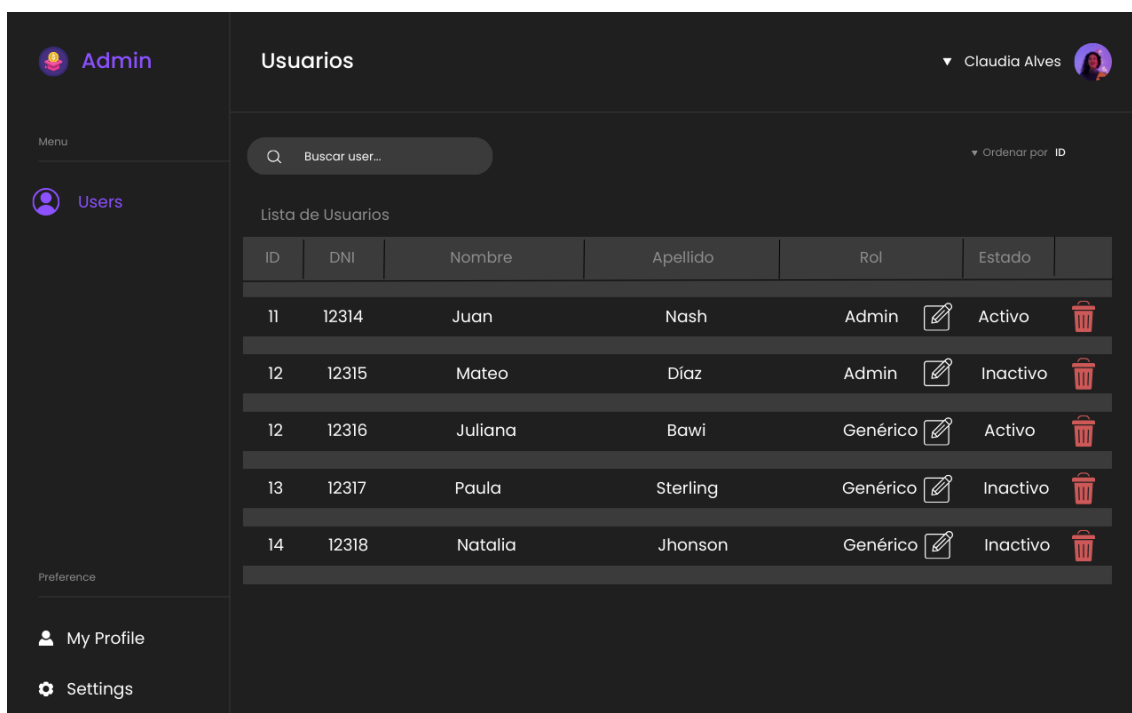
Altura - 200

Preference

My Profile

Settings

Figura 15



Usuarios

▼ Claudia Alves

Menu

Users

Buscar user...

Ordenar por ID

Lista de Usuarios

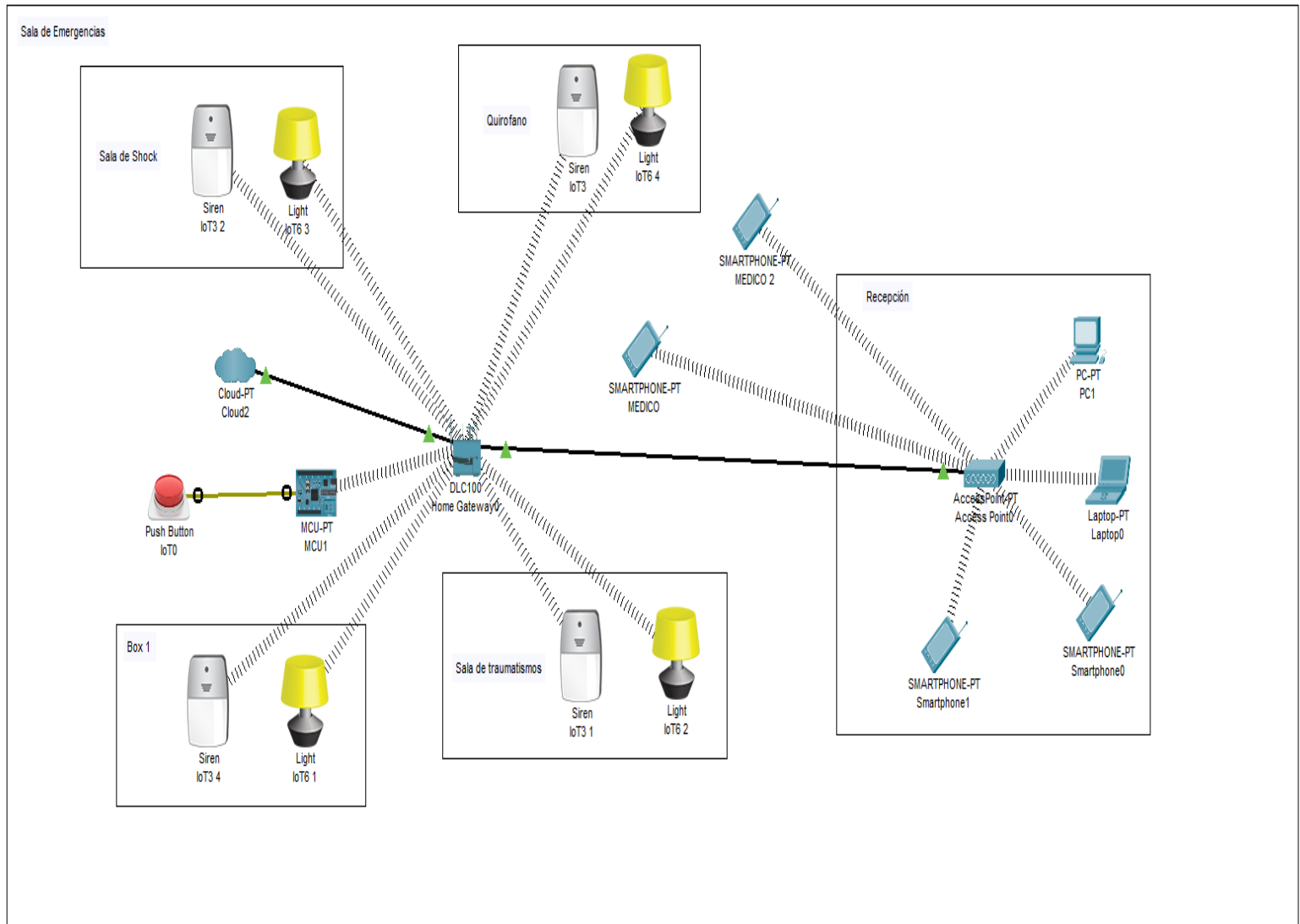
ID	DNI	Nombre	Apellido	Rol	Estado	
11	12314	Juan	Nash	Admin	Activo	 
12	12315	Mateo	Díaz	Admin	Inactivo	 
12	12316	Juliana	Bawi	Genérico	Activo	 
13	12317	Paula	Sterling	Genérico	Inactivo	 
14	12318	Natalia	Jhonson	Genérico	Inactivo	 

Preference

My Profile

Settings

Figura 16



9. Registro de experiencia

División de tareas y gestión del tiempo:

Con respecto a la división grupo se dividió en dos subgrupos, uno para trabajar en la interfaz de usuario y otro para trabajar en las funcionalidades, en donde todos contribuimos con la conformación del informe y nuevas ideas subyacentes en ese transcurso.

Nuestro grupo ya tenía breves experiencias para la realización de proyectos de este calibre, pero gracias a la gestión del tiempo hecha en Trello, y la división de grupos se nos facilitó mucho más a la hora de confeccionar el sistema.

Dificultades:

Una de las principales dificultades que enfrentó el grupo fue la definición de los requisitos funcionales del sistema. El grupo tuvo que entrevistar a personal médico y a representantes de diferentes centros médicos para comprender las necesidades de los usuarios.

Solución:

Para definir los requisitos funcionales del sistema, el grupo utilizó una metodología de desarrollo centrada en el usuario. Para implementar la lógica del sistema, el grupo utilizó un enfoque modular para facilitar el desarrollo y el mantenimiento del sistema.

Conclusiones

- **Desafíos:** El trabajo grupal puede ser desafiante por varias razones. Por ejemplo, puede ser difícil coordinar el trabajo de personas con diferentes habilidades y estilos de trabajo. También puede ser difícil llegar a acuerdos y tomar decisiones.
- **Beneficios:** El trabajo grupal también tiene muchos beneficios. Por ejemplo, puede ayudar a las personas a aprender de los demás, a desarrollar habilidades de comunicación y colaboración, y a lograr objetivos que serían imposibles de alcanzar de forma individual.

10. Bibliografía

- ☑ **01/09/2023** - Curso Python Django -
📺 Django, Curso de Django para Principiantes - **Autor:** Fazt.

- ☑ **02/09/2023** - Curso Python Django Backend -
📺 Curso de PYTHON desde CERO para BACKEND - **Autor:** MoureDev by Brais Moure.

- ☑ **03/09/2023** - Python Django Documentation -
<https://docs.djangoproject.com/en/4.2/> - **Autor:** Django.

- ☑ **23/09/2023** - Tutorial Flutter Flow -
📺✅ Flutterflow desde 0 #1 - Primera App: Contador. Diagrama de flujo. M...
- **Autor:** IvanSD76